SHORT Answers

700776646

3)

a) Non-linear activation functions are needed b'cos they let Neural networks learn complex patterns. without them, the network would behave like a simple linear model and wouldn't capture real-world relationships.

b) Deep N/w can learn multiple levels of features, starting from simple patterns and building up to more complex ones. shallow models like logistic regression cannot represent such layered and detailed patterns.

2) RNN families

a) mapping tasks to RNN patterns

→ Next word prediction:- many-to-many aligned.
each input word help predict the next word at every step in the sequence..

→ Sentiment of a Sentence :- many-to-one
The entire sequence is processed to produce a single output (sentiment)

→ NER (Named entity Recognition): Many-to-many-aligned.
each word in the i/p sequence gets a corresponding label.

→ Machine translation:- Many-to-many unaligned.
input and output sequence differen in length and are not step-by-step aligned.

b) unrolling an RNN over time creates a sequence of connected steps allowing gradients to flow backward through time while using the same weights at each step.

c) * Advantage :- reduces the number of parameters and improves generalisation.

* Limitation :- makes it harder to capture long-range dependencies.

2) Vanishing gradients & remedies :-

a) Gradients shrink through many time steps, so RNN's fail to learn long range dependencies.

b) LSTM :- cell state gives a more stable gradient path.

GRU :- gating reduces repeated multiplication and preserves gradients.

c) Gradient clipping - prevents gradients from exploding or collapsing during backpropagation.

3) LSTM gates & cell state :-

a) forget gate (sigmoid) :- decides the past info to drop.

input gate (sigmoid + tanh) :- decides what new info to add.

output gate (sigmoid) :- controls what part of the cell state becomes hidden state.

b) because the cell state uses additive updates, allowing gradients to flow without repeated multiplication.

c) "what to remember" = update cell state, "what to expose" = output gate controlling hidden state.

4) Self attention

a) Q = what were looking for ; K = what tokens offers ; V = content to mix.

b) ~~Attention~~ $(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

c) To keep dot products from becoming too large and destabilizing Softmax.

5) Multi head Attention & Residuals:-

a) multiple heads let the model learn different relations in parallel,

b) Residuals help gradient flow, layer Norm stabilize training dynamics

c) one head may track subject-verb agreement.

6) Encoder - decoder with Masking:-

a) Masking prevents the decoder from seeing future tokens and cheating.

b) Encoder self-attention attends with the input, Cross-attention let the decoder attend to encoder outputs.

c) it generates one token at a time, feeding each new output back as input for the next step.

NLP - H.W. 4

700776646

Q)6

given $x_1 = 2$, $x_2 = 1$, $x_3 = 3$

Weights

$$A = \begin{bmatrix} 0.2 & -0.5 \\ 0.1 & 0.3 \\ -0.2 & 0.8 \\ 0.4 & -0.6 \end{bmatrix}, \quad B = \begin{bmatrix} 0.7 \\ -1.2 \\ 0.5 \end{bmatrix}$$

(a) Hidden layer computation

Add bias input = 1

$$z_1 = (2)(0.2) + (1)(0.1) + 3(-0.2) + 1(0.4) = 0.3.$$

$$z_2 = (2)(-0.5) + (1)(0.3) + 3(0.8) + 1(0.6) = 1.1$$

Apply sigmoid :-

$$h_1 = \sigma(0.3) \approx 0.574$$

$$h_2 = \sigma(1.1) \approx 0.750.$$

(b) output computation :-

$$z = (0.574)(0.7) + 0.750(-1.2) + (1)(0.5)$$

$$z \approx 0.402 - 0.900 + 0.5 = 0.002.$$

$$y = \sigma(0.002) \approx 0.5005.$$

(c) Binary cross entropy loss ($t = 1$)

$$L = -\log(y)$$

$$L = -\log(0.5005) \approx 0.692$$

7) XOR with ReLU N/w.

Ans:- given

$$h_1 = ReLU(x_1 + x_2), \quad h_2 = ReLU(x_1 + x_2 - 1)$$

$$h_3 = ReLU(2x_1 - x_2)$$

$$y = ReLU(h_1 - 2h_2 + h_3)$$

a) compute outputs:-

(0,0) $h_1 = 0, h_2 = 0, h_3 = 0$.
$y = 0$.

(1,0) $h_1 = 0, h_2 = 0, h_3 = 2$
$y = 1 - 0 + 2 = 3$.

(1,1) :- $h_1 = 2, h_2 = 1, h_3 = 1$

$y = 2 - 2 + 1 = 1$

(0,1) : $y = 1$.

b) decision boundary:-
The additional hidden unit creates extra linear regions, making the boundary more complex compared to original XOR network.

c) XOR check
The N/w doesnt compute XOR exactly.
(1,1) gives 1 instead of 0.
(1,0) gives 3 instead of 1.
The added hidden unit changes the function. So the n/w no longer represents XOR correctly.

8) Decision boundary and Misclassification.

given $y = \begin{cases} 1 & \text{if } w_1 x_1 + w_2 x_2 + b > 0 \\ 0 & \text{otherwise} \end{cases}$

$w_1 = 1, w_2 = -2, b = 1.$

dataset $(2,1) \to 1$, $(1,3) \to 0$, $(3,2) \to 1$, $(0,1) \to 0$.

Ans)
a) Decision boundary

$w_1 x_1 + w_2 x_2 + b = 0$

$$\boxed{x_1 - 2x_2 + 1 = 0}$$

This is a straight line.

Ans) b) Classification.

$(2,1)$: $2 - 2 + 1 = 1 > 0$, $y = 1$.

$(1,3)$: $1 - 6 + 1 = -4 < 0 \Rightarrow y = 0$.

$(3,2)$: $3 - 4 + 1 = 0 \Rightarrow y = 0$ (wrong, expected 1)

$(0,1)$: $0 - 2 + 1 = -1 < 0 \Rightarrow y = 0$

$\boxed{\text{Misclassified point } (3,2)}$

Ans) c) perceptron loss.

no of Mistake: 1.

Ans) d) weight update ($\eta = 0.5$)

update rule:- $w = w + \eta(t - y)x$

$b = b + \eta(t - y)$

for point $(3,2)$: $t = 1, y = 0$, $\Rightarrow (t - y) = 1$.

updated weights

$w_1 = 1 + 0.5(3) = 2.5$

$w_2 = -2 + 0.5(2) = -1$

$b = 1 + 0.5 = 1.5$

finally decision boundary: $x_1 - 2x_1 + 1 = 0$.